HEXLOGIC

IoT & EMBEDDED PENETRATION TESTING

"OWASP IoTGoat firmware v1.0"

Document Information

Project Name	IoT & Embedded Penetration Testing
Document Title	IoT & Embedded Penetration Test Report
Document Type	Final Report
Classification	Confidential & Restricted
Created By	HexLogic Offensive Security

Document History

Version	Author	Date	Change Description
1.0	HexLogic	30 May 2025	Final Report

Table of Contents

1 Introduction

This report describes the proceedings and results of the IoT and embedded device assessment conducted by HexLogic against OWASP IoTGoat firmware v1.0. It enlists the findings identified during the engagement, their business impact and recommended remediation.

1.1 Objective

The objective of the assessment was to evaluate the security posture of OWASP IoTGoat firmware v1.0 and to provide a final security review report comprising a remediation strategy and recommendations to help mitigate the identified vulnerabilities and risks. The assessment was conducted presuming the malicious intent of an external adversary in order to identify associated business risk and its impact.

1.2 Assessment Duration

Test Duration	15 May 2025 – 23 May 2025
Total Days	7 Days
Note	First Test

1.3 Team Contact Details

Name	HexLogic
Contact	security@hexlogic.io
Role	Penetration Testing Team

2 Scope

This section defines the scope and boundaries of the engagement.

2.1 Target Scope

- IoTGoat-raspberry-pi2.img (OWASP IoTGoat firmware image)
- Device web interface and exposed network services

2.2 Assessment Attributes

Starting Vector	Network and physical (firmware analysis)
Target Criticality	Assessment Environment
Assessment Nature	Cautious & calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable

2.3 Assessment Type and Methods

The assessment was performed in a grey-box manner, primarily manual in nature while using a limited, necessary tool-set to make the process more efficient, preserving the imitation of a real-world adversary. Testing covered firmware extraction and analysis, hardcoded secret discovery, network service assessment, web-interface testing, update-integrity review and hardware-interface inspection.

2.4 Methodology & Risk Classification

The following risk classification is used throughout this report. Findings are rated using CVSS v3.1 and business context.

Critical	Trivially exploitable issues leading to full system or data compromise; remediate immediately.
High	Serious issues allowing significant unauthorised access or impact; remediate as a priority.
Medium	Issues exploitable under certain conditions, often combined with others; schedule remediation.
Low	Limited-impact weaknesses and defence-in-depth gaps; address during routine hardening.
Info	No direct threat, but may reveal sensitive information useful to an attacker.

3 Executive Summary

The target in scope was reviewed for the adequacy of its existing security controls in accordance with the OWASP IoT Top 10 and the OWASP Firmware Security Testing Methodology (FSTM) and industry best practices. The aim of the testing was to establish whether the target could be compromised to allow unauthorised access to sensitive data or systems.

A total of 8 vulnerabilities were identified during the assessment, comprising 2 Critical, 2 High, 3 Medium and 1 Low-risk findings. The firmware contains hardcoded backdoor credentials and a command-injection flaw in the web interface that allow unauthenticated device takeover; these critical issues must be remediated immediately, alongside the weak update, exposed-interface and outdated-component findings.

3.1 Graphical Representation of Vulnerabilities

The table below summarises the overall risk identified during the assessment.

Critical	High	Medium	Low	Total
2	2	3	1	8

4 Compliance – Detailed Summary

The final risk value of each vulnerability is derived by considering the likelihood of exploitation and the impact on the business of the assessed target.

No	Findings	Severity	CVSS 3.1	Status
1	Hardcoded Backdoor Credentials in Firmware	Critical	9.8	Open
2	Command Injection in Device Web Interface	Critical	9.8	Open
3	Telnet Service Enabled with Default Credentials	High	8.1	Open
4	Insecure Firmware Update — No Signature Verification	High	7.5	Open
5	Exposed UART Serial Console Without Authentication	Medium	6.8	Open
6	Hardcoded TLS Private Key in Firmware	Medium	6.5	Open
7	Outdated Components with Known Vulnerabilities	Medium	5.9	Open
8	Sensitive Information Stored in Firmware	Low	3.7	Open

5 Detailed Technical Summary

This section provides the technical detail for each finding, including a comprehensive description, business impact, reproduction steps, proof of concept and remediation guidance.

5.1 Hardcoded Backdoor Credentials in Firmware

Critical	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Open

Description

Extraction and analysis of the firmware reveals hardcoded credentials baked into the device image, including an undocumented support account with a weak, crackable password hash. Because the same firmware is deployed across every unit, these credentials grant an attacker authenticated access to any device of this model. Hardcoded credentials cannot be changed by end users and provide a reliable, fleet-wide backdoor.

Affected Component

/etc/shadow, /etc/passwd (extracted firmware)

Impact

- Authenticated administrative access to every device running this firmware.
- Full control of device functionality and any connected systems or data.
- A fleet-wide compromise vector that persists across reboots and user password changes.

Steps to Reproduce

1. Extract the firmware filesystem with binwalk.
2. Locate and read /etc/passwd and /etc/shadow from the extracted root filesystem.
3. Crack the recovered hash offline and authenticate to the device with the credentials.

Proof of Concept

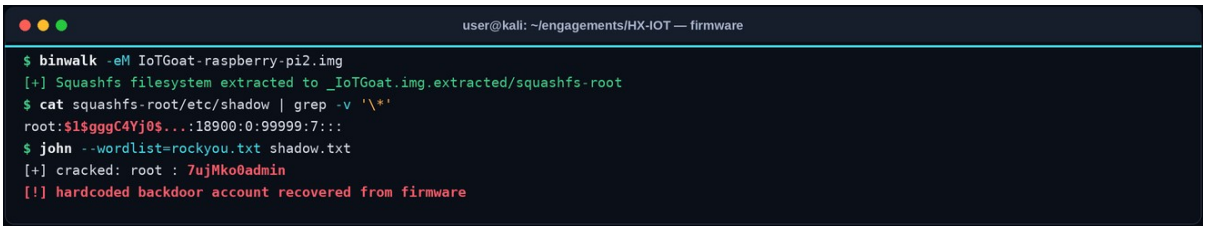

Figure 1. Firmware extraction reveals a hardcoded account whose password hash is cracked offline.

Recommendations

- Remove all hardcoded and default credentials from firmware images.
- Provision unique, random per-device credentials at manufacture and force a change on first use.
- Use strong, salted password hashing and store secrets in secure hardware where available.
- Establish a firmware secret-scanning gate in the build pipeline.

References

- OWASP IoT Top 10 — I1 Weak/Guessable/Hardcoded Passwords
- CWE-798

5.2 Command Injection in Device Web Interface

Critical	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Open

Description

The diagnostics feature of the device web interface passes a user-supplied host value to a shell command without sanitisation. By injecting shell metacharacters, an attacker can execute arbitrary operating-system commands on the device with the privileges of the web server, which on embedded devices is typically root. The endpoint is reachable without authentication on this firmware.

Affected Endpoint

POST /cgi-bin/diagnostics (ping/host field)

Impact

- Unauthenticated remote command execution as root on the device.
- Complete device takeover, including pivoting to the local network the device sits on.
- Ability to brick, backdoor or enrol the device into a botnet.

Steps to Reproduce

1. Identify the diagnostics endpoint that performs a ping/host lookup.
2. Inject a shell metacharacter and command into the host field.
3. Observe the command output returned, confirming execution.

Proof of Concept

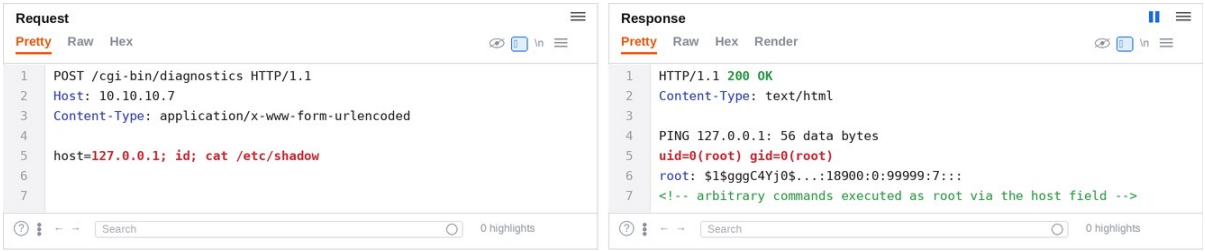

Figure 2. Shell metacharacters in the diagnostics host field execute arbitrary commands as root.

Recommendations

- Never pass user input to a shell; use safe, parameterised APIs for network operations.
- Apply strict allow-list input validation (valid hostname/IP only) and reject metacharacters.
- Run web services under a least-privilege account, not root, and require authentication.
- Add a tested security regression for command-injection on all input-handling endpoints.

References

- OWASP — Command Injection
- CWE-78

5.3 Telnet Service Enabled with Default Credentials

High	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L	8.1	Open

Description

The device exposes a Telnet service that is enabled by default and accepts the firmware's default credentials. Telnet additionally transmits all data, including the login, in cleartext. This combination provides a trivial, well-known remote administration backdoor of the kind abused at scale by IoT botnets such as Mirai.

Affected Service

10.10.10.7:23 (Telnet)

Impact

- Trivial remote shell access using default credentials over an unauthenticated, cleartext protocol.
- Mass exploitation potential consistent with known IoT botnet behaviour.
- Interception of credentials by any on-path attacker.

Steps to Reproduce

1. Confirm Telnet is listening on the device.
2. Authenticate using the firmware's default credentials.
3. Obtain an interactive shell and confirm privileges.

Proof of Concept

Figure 3. Telnet is enabled by default and grants a root shell using the firmware's default credentials.

Recommendations

- Disable Telnet entirely; provide management only over authenticated, encrypted channels (SSH).
- Remove default credentials and require per-device secrets.
- Disable unused services in the firmware build and close their ports by default.
- Apply network segmentation so management services are not internet-exposed.

References

- OWASP IoT Top 10 — I1/I2
- CWE-319

5.4 Insecure Firmware Update — No Signature Verification

High	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H	7.5	Open

Description

The device accepts firmware updates without verifying a cryptographic signature or performing integrity validation against a trusted key. An attacker who can deliver an update image, for example via a compromised update channel or local access, can install malicious firmware that persists across reboots and resets, giving durable, low-level control of the device.

Affected Component

Firmware update mechanism

Impact

- Installation of attacker-controlled firmware that persists across factory resets.
- Durable, stealthy compromise of the device and any data it handles.
- Supply-chain risk if the update channel is compromised.

Steps to Reproduce

1. Analyse the update routine and confirm no signature check is performed.
2. Build a modified firmware image with an added payload.
3. Upload the unsigned image and confirm it is accepted and applied.

Proof of Concept

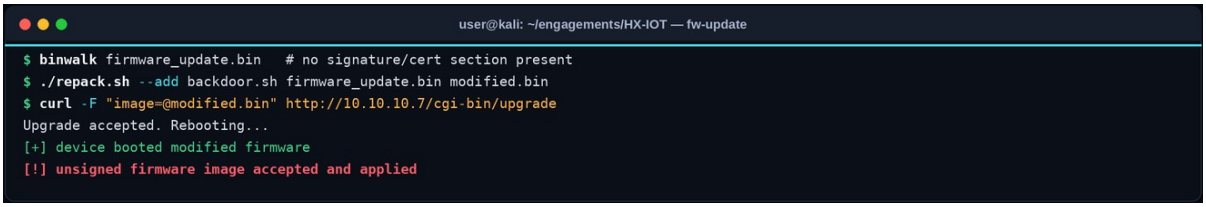

Figure 4. A modified, unsigned firmware image is accepted and applied by the device's update mechanism.

Recommendations

- Cryptographically sign firmware and verify the signature against a hardware root of trust before applying.
- Implement secure/verified boot and anti-rollback protection.
- Deliver updates over authenticated, encrypted channels and validate image integrity.
- Fail closed on verification errors.

References

- OWASP IoT Top 10 — I7 Insecure Update Mechanism
- CWE-347

5.5 Exposed UART Serial Console Without Authentication

Medium	CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	6.8	Open

Description

The device exposes an accessible UART serial header that provides an unauthenticated root shell and bootloader access. An attacker with brief physical access can connect to the header, interrupt the boot process, and gain low-level control of the device, extract secrets and modify the boot configuration. Debug interfaces left enabled in production are a common embedded weakness.

Affected Component

On-board UART header

Impact

- Unauthenticated root shell and bootloader access via physical UART.
- Extraction of firmware, keys and credentials from a single physical device.
- Modification of boot parameters to disable protections.

Steps to Reproduce

1. Identify the UART pins (TX/RX/GND) on the board.
2. Connect a USB-to-serial adapter at the correct baud rate.
3. Observe the boot log and an unauthenticated root shell or bootloader prompt.

Proof of Concept

Figure 5. Connecting to the on-board UART yields an unauthenticated root shell and bootloader access.

Recommendations

- Disable serial console login in production or require authentication on the console.
- Lock the bootloader, disable interactive prompts and enable secure boot.
- Remove or physically protect debug headers on production hardware.
- Store secrets in a secure element so console access does not yield key material.

References

- OWASP FSTM — Hardware Interfaces
- CWE-1191

5.6 Hardcoded TLS Private Key in Firmware

Medium	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N	6.5	Open

Description

The firmware ships with an embedded TLS private key that is identical across every device of this model. An attacker who extracts the key from one device can decrypt or impersonate the TLS communications of any other device using the same firmware, undermining the confidentiality and authenticity that TLS is meant to provide.

Affected Component

/etc/ssl/private/ (extracted firmware)

Impact

- Decryption and impersonation of device TLS traffic across the entire fleet.
- Man-in-the-middle attacks against device-to-cloud and management communications.
- Loss of trust in the device's identity.

Steps to Reproduce

1. Extract the firmware filesystem.
2. Locate the embedded TLS private key.
3. Demonstrate that the same key is present on other devices of the model.

Proof of Concept

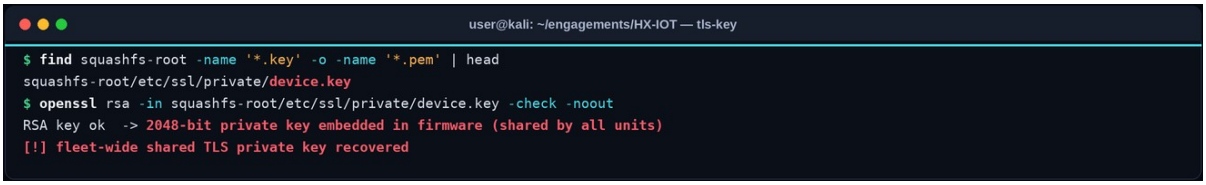

Figure 6. A shared TLS private key is recovered from the firmware, allowing fleet-wide decryption and impersonation.

Recommendations

- Generate unique TLS key pairs per device at provisioning, ideally inside a secure element.
- Never embed shared private keys in firmware images.
- Use a device-identity/PKI scheme with per-device certificates and revocation.
- Rotate and revoke any keys exposed in shipped firmware.

References

- OWASP FSTM — Analysing Filesystem Contents
- CWE-321

5.7 Outdated Components with Known Vulnerabilities

Medium	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:L	5.9	Open

Description

The firmware bundles outdated versions of core components such as BusyBox, Dropbear SSH and OpenSSL, several of which are affected by publicly disclosed vulnerabilities. Embedded devices frequently ship and remain in the field with stale components, and the lack of a maintenance process means these known issues persist for the life of the device.

Affected Component

BusyBox, Dropbear, OpenSSL (firmware)

Impact

- Exposure to publicly known vulnerabilities in bundled components.
- Increased likelihood of remote exploitation as public exploits exist.
- Long-lived risk due to infrequent device patching.

Steps to Reproduce

1. Enumerate component versions from the extracted firmware (strings/version files).
2. Cross-reference each version against vulnerability databases.
3. Confirm the presence of components with known CVEs.

Proof of Concept

Figure 7. Firmware analysis reveals outdated BusyBox, Dropbear and OpenSSL versions with known CVEs.

Recommendations

- Update all bundled components to current, supported versions and rebuild the firmware.
- Maintain an SBOM for firmware and monitor it for newly disclosed vulnerabilities.
- Establish a firmware patch/update lifecycle with defined support timelines.
- Remove components that are not required.

References

- OWASP IoT Top 10 — I5 Use of Insecure or Outdated Components
- CWE-1104

5.8 Sensitive Information Stored in Firmware

Low	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	3.7	Open

Description

Configuration files within the firmware contain sensitive information such as internal hostnames, API endpoints, Wi-Fi configuration and references to cloud services. While individually low impact, this information aids an attacker in mapping the device's ecosystem and planning attacks against the backend infrastructure.

Affected Component

Configuration files (extracted firmware)

Impact

- Disclosure of internal endpoints and configuration aiding further attacks.
- Mapping of the device's backend and cloud ecosystem.
- Support for targeted attacks against supporting infrastructure.

Steps to Reproduce

1. Search the extracted filesystem for configuration and secret patterns.
2. Review the discovered files for sensitive references.
3. Document the disclosed endpoints and configuration.

Proof of Concept

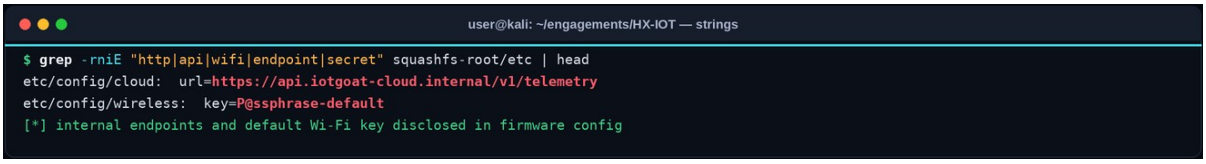

Figure 8. Firmware configuration files disclose internal endpoints and a default Wi-Fi passphrase.

Recommendations

- Remove sensitive configuration and secrets from shipped firmware.
- Provision device-specific configuration securely at runtime rather than baking it in.
- Avoid embedding internal infrastructure details that aid reconnaissance.
- Review firmware contents for sensitive data before release.

References

- OWASP FSTM — Filesystem Analysis
- CWE-312

THANK YOU

HEXLOGIC